

CHAPTER 1 · SUB-CHAPTER 1.2

Collections Framework & List Implementations

Framework Architecture · ArrayList · LinkedList · CopyOnWriteArrayList
Iteration Patterns · ConcurrentModificationException · Immutable Lists
Performance Comparison · Advanced Internals · Interview Q&A

~20 Pages

52+ Q&A

Adv. Questions

Interview Ready

■ Purple [ADV] questions indicate advanced / senior-level depth

Section 1 — Collections Framework Architecture

1.1 Interface Hierarchy

The Java Collections Framework (JCF) is a unified architecture for representing and manipulating groups of objects. At its root is `java.lang.Iterable`, which enables the for-each loop. `java.util.Collection` extends `Iterable` and is the root interface for all single-element containers. `Map` sits in its own hierarchy and deliberately does NOT extend `Collection` because a `Map` is not a collection of elements — it is a collection of key-value mappings. Mixing the two would break the semantics of methods like `add(E e)`.

```
// Full Interface Hierarchy (ASCII diagram)
java.lang.Iterable<T>
  ■■■ java.util.Collection<E>
    ■■■ java.util.List<E> (ordered, indexed, allows duplicates)
      ■ ■■■ ArrayList (resizable array)
      ■ ■■■ LinkedList (doubly-linked, also Deque)
      ■ ■■■ Vector (legacy)
      ■ ■■■ CopyOnWriteArrayList (thread-safe, snapshot iterator)
      ■
    ■■■ java.util.Set<E> (no duplicates)
      ■ ■■■ HashSet (hash table, O(1) ops)
      ■ ■■■ LinkedHashSet (insertion-ordered hash table)
      ■ ■■■ java.util.SortedSet<E> → NavigableSet<E>
      ■ ■■■ TreeSet (Red-Black tree, O(log n) ops)
      ■
    ■■■ java.util.Queue<E> (FIFO semantics)
    ■■■ PriorityQueue (heap-based)
    ■■■ java.util.Deque<E> (double-ended queue)
    ■■■ ArrayDeque (resizable circular array)
    ■■■ LinkedList (also implements List!)

// Map hierarchy (SEPARATE – does NOT extend Collection)
java.util.Map<K,V>
  ■■■ HashMap (hash table, O(1) avg)
  ■■■ LinkedHashMap (insertion/access-ordered)
  ■■■ Hashtable (legacy, synchronized)
  ■■■ java.util.SortedMap<K,V> → NavigableMap<K,V>
  ■■■ TreeMap (Red-Black tree, O(log n))
```

1.2 Key Interfaces Summary

Interface	Extends	Key Contract	Notable Methods
Collection	Iterable	Basic group of elements	add, remove, contains, size, iterator, stream
List	Collection	Ordered, indexed, duplicates OK	get(i), set(i,e), add(i,e), indexOf, subList, listIterator
Set	Collection	No duplicates	add, contains, remove (same as Collection but no duplicates)
Queue	Collection	FIFO ordering; offer/poll/peek	offer, poll, peek, add, remove, element
Deque	Queue	Double-ended queue	addFirst/Last, removeFirst/Last, peekFirst/Last, offerFirst/Last
NavigableSet	SortedSet	Navigation ops on sorted set	floor, ceiling, lower, higher, pollFirst, pollLast, descendingSet
NavigableMap	SortedMap	Navigation ops on sorted map	floorKey, ceilingKey, lowerEntry, higherEntry, subMap, headMap, tailMap

1.3 Abstract Base Classes

The JCF provides abstract skeleton implementations to simplify custom collection creation. These use the Template Method pattern — concrete subclasses implement only a few primitive operations.

Abstract Class	Implements	Must Override	Provides Free
AbstractCollection	Collection	iterator(), size()	contains, toArray, add (throws), remove, clear

AbstractList	List + AbstractCollection	get(i), size()	iterator, listIterator, indexOf, subList, add (throws)
AbstractSequentialList	AbstractList	listIterator(i), size()	get(i) via listIterator — for sequential access
AbstractSet	Set + AbstractCollection	iterator(), size()	equals, hashCode based on set semantics
AbstractMap	Map	entrySet()	get, containsKey, size, put (throws), keySet, values
AbstractQueue	Queue + AbstractCollection	offer, peek, poll, iterator, size	add, remove, element, clear

KEY INSIGHT: Why doesn't Map extend Collection? A Map.put(k,v) takes two arguments; Collection.add(e) takes one. Forcing Map into the Collection hierarchy would require either breaking the add() contract or leaving it unsupported. The designers deliberately kept Map as a parallel hierarchy.

Section 2 — ArrayList Deep Dive

2.1 Internal Structure & Capacity Growth

ArrayList is backed by a private **Object[] elementData** array. When created with the no-arg constructor it starts with an empty array (DEFAULTCAPACITY_EMPTY_ELEMENTDATA) and lazily grows to capacity 10 on the first add(). When the array is full, it grows by **1.5× (newCapacity = oldCapacity + oldCapacity >> 1)**. This gives amortised O(1) appends. The 1.5× factor (vs 2×) trades slightly more frequent resizes for lower memory waste.

```
// Simplified growth logic (from JDK source)
private Object[] grow(int minCapacity) {
    int oldCapacity = elementData.length;
    if (oldCapacity > 0 || elementData != DEFAULTCAPACITY_EMPTY_ELEMENTDATA) {
        int newCapacity = ArraysSupport.newLength(oldCapacity,
            minCapacity - oldCapacity, // min growth
            oldCapacity >> 1); // preferred growth = 50%
        return elementData = Arrays.copyOf(elementData, newCapacity);
    } else {
        return elementData = new Object[Math.max(DEFAULT_CAPACITY, minCapacity)];
    }
    // DEFAULT_CAPACITY = 10
}
// Arrays.copyOf delegates to System.arraycopy (native, JNI) – very fast bulk copy
```

2.2 Time Complexity

Operation	Time Complexity	Notes
get(index)	O(1)	Direct array access — random access
set(index, e)	O(1)	Direct array write
add(e) — append	Amortised O(1)	Occasionally O(n) on resize; amortised constant
add(i, e) — mid	O(n)	Must shift elements i..size-1 right by 1
remove(index)	O(n)	Must shift elements after index left by 1
remove(Object o)	O(n)	Linear scan + shift
contains(o)	O(n)	Linear scan via equals()
indexOf(o)	O(n)	Linear scan from front
lastIndexOf(o)	O(n)	Linear scan from back
sort(Comparator)	O(n log n)	Uses TimSort (merge+insertion hybrid)
iterator()	O(1)	Returns Itr inner class instance
subList(from,to)	O(1)	Returns a view, NOT a copy
toArray()	O(n)	Arrays.copyOf — copies all active elements

2.3 Common Operations Code Examples

```
import java.util.*;

List<String> list = new ArrayList<>(16); // pre-size to avoid resize

// Basic add / get / set / remove
list.add("apple");
list.add("banana");
list.add("cherry");
list.add(1, "avocado"); // insert at index 1 – O(n)
String s = list.get(0); // "apple"
list.set(2, "blueberry"); // replace index 2
list.remove(0); // remove by index – returns removed element
```

```

list.remove("blueberry"); // remove by value – returns boolean
// Bulk operations
List<String> more = List.of("date", "elderberry");
list.addAll(more);
list.addAll(1, more); // insert collection at position
// Search
int idx = list.indexOf("banana");
int last = list.lastIndexOf("banana");
boolean has = list.contains("cherry");
// subList – RETURNS A VIEW, not a copy
List<String> view = list.subList(1, 3);
view.set(0, "MODIFIED"); // also modifies the parent list!
// view.add("x"); // structural mod on view is reflected in parent
// Sort
list.sort(Comparator.naturalOrder());
list.sort(Comparator.reverseOrder());
list.sort(Comparator.comparingInt(String::length).thenComparing(Comparator.naturalOrder()));
// replaceAll – in-place transformation
list.replaceAll(String::toUpperCase);
// removeIf – conditional removal (safe, no CME)
list.removeIf(str -> str.startsWith("A"));
// Iterator
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    String val = it.next();
    if (val.equals("BANANA")) it.remove(); // safe removal during iteration
}
// ListIterator – bidirectional + add/set
ListIterator<String> li = list.listIterator(list.size());
while (li.hasPrevious()) {
    System.out.println(li.previous());
}

```

2.4 Factory Methods Comparison

Factory Method	Mutable?	Null Elements?	Backed by Array?	Notes
<code>new ArrayList<>()</code>	Yes (fully)	Yes	Own copy	General-purpose mutable list
<code>Arrays.asList(a,b,c)</code>	Partially (set only)	Yes	Yes — the original array	Fixed-size: add/remove throw <code>UnsupportedOperationException</code>
<code>List.of(a,b,c)</code>	No	No — NPE on null	No (ImmutableCollections)	Java 9+; completely immutable; no duplicates requirement for Set equiv
<code>List.copyOf(col)</code>	No	No — NPE on null	No	Java 10+; makes defensive copy then wraps immutably
<code>Collections.singletonList(e)</code>	No (view)	Yes	No	Single element; throws on add/remove
<code>Collections.unmodifiableList(list)</code>	No (view)	Yes	Backed by original	Original still mutable — changes visible through this wrapper

Section 3 — LinkedList Deep Dive

3.1 Internal Structure

LinkedList is a **doubly-linked list** where each element is stored in a **Node<E>** object containing references to the previous node, next node, and the element value. It implements both **List<E>** and **Deque<E>**, making it usable as a list, stack, or queue. Head and tail (first/last) references are maintained for O(1) access at both ends.

```
// Simplified Node structure (from JDK source)
private static class Node<E> {
    E item;
    Node<E> next;
    Node<E> prev;
    Node(Node<E> prev, E element, Node<E> next) {
        this.item = element;
        this.next = next;
        this.prev = prev;
    }
}

// Each node: 3 object references = ~24 bytes overhead per element (64-bit JVM, no compressed oops)
// vs ArrayList: ~8 bytes per element (one object reference in the array)
// Memory layout
// first --> [null | "A" | *] --> [* | "B" | *] --> [* | "C" | null] <-- last
// Node<E> Node<E> Node<E>
```

3.2 Time Complexity

Operation	Time Complexity	Notes
addFirst(e) / addLast(e)	O(1)	Pointer update at head/tail
removeFirst() / removeLast()	O(1)	Pointer update at head/tail
get(index)	O(n)	Must traverse from nearest end
add(index, e)	O(n)	O(n) to find node + O(1) pointer update
remove(index)	O(n)	O(n) to find node + O(1) pointer update
contains(o)	O(n)	Linear scan via equals()
iterator()/next()	O(1) per step	Follows next reference
listIterator add/remove	O(1)	At current position — major advantage
size()	O(1)	Maintained as field
peek() / peekFirst()	O(1)	Returns first.item without removing
poll() / pollFirst()	O(1)	Returns and removes first element

3.3 Deque Operations

```
Deque<String> deque = new LinkedList<>();
// Stack operations (LIFO)
deque.push("a"); // = addFirst
deque.push("b");
String top = deque.pop(); // = removeFirst => "b"
// Queue operations (FIFO)
deque.offer("x"); // = addLast
deque.offer("y");
String head = deque.poll(); // = removeFirst => "x"
// Deque-specific
deque.addFirst("START");
```

```
deque.addLast("END");
deque.peekFirst(); // inspect without removing
deque.peekLast();
deque.offerFirst("X"); // offer = add but returns false instead of throwing
deque.offerLast("Y");
deque.removeFirst(); // throws NoSuchElementException if empty
deque.pollFirst(); // returns null if empty (safe)
deque.descendingIterator(); // iterate from tail to head
// Note: Prefer ArrayDeque over LinkedList for pure Deque use
// ArrayDeque has better cache locality and no node allocation overhead
```

WHEN TO USE LinkedList: (1) You need a Deque AND a List simultaneously with the exact same object reference. (2) You use ListIterator.add() or .remove() heavily in the middle of iteration — $O(1)$ per op vs $O(n)$ for ArrayList. OTHERWISE: Prefer ArrayList for List use and ArrayDeque for Deque/Queue use. LinkedList's poor cache locality (scattered Node objects on heap) kills performance for sequential access.

Section 4 — CopyOnWriteArrayList

4.1 Design & Semantics

CopyOnWriteArrayList (`java.util.concurrent`) achieves thread safety without read locks by making a fresh copy of the backing array on every mutating operation (`add`, `set`, `remove`). The internal array reference is declared **volatile** so all threads see the latest write without locking. Iterators capture a snapshot of the array at creation time and never throw `ConcurrentModificationException`.

```
// Simplified internals (from JDK source)
public class CopyOnWriteArrayList<E> {
    private transient volatile Object[] array; // volatile ensures visibility
    // Read – lock-free, just reads the volatile array reference
    public E get(int index) {
        return elementAt(getArray(), index); // no lock needed!
    }
    // Write – synchronized, copies the entire array
    public boolean add(E e) {
        synchronized (lock) {
            Object[] es = getArray();
            int len = es.length;
            es = Arrays.copyOf(es, len + 1); // O(n) copy
            es[len] = e;
            setArray(es); // atomic volatile write
            return true;
        }
    }
    // Iterator – snapshot semantics
    public Iterator<E> iterator() {
        return new COWIterator<E>(getArray(), 0); // captures current array
    }
    // COWIterator: no remove(), no CME, operates on immutable snapshot
}
```

4.2 Use Cases and Trade-offs

Aspect	CopyOnWriteArrayList	Collections.synchronizedList	ArrayList (unsynchronized)
Read performance	Excellent — lock-free	Moderate — locks on every read	Excellent — lock-free
Write performance	Poor — O(n) copy	Good — O(1) lock+write	Good — amortised O(1)
Iterator thread safety	Safe (snapshot)	Must manually synchronize block	Not thread-safe (CME)
ConcurrentModificationException	Never	Yes if not synchronized	Yes (fail-fast)
Memory	2x during write	1x	1x
Best use case	Read-heavy, rare writes; event listeners	General synchronized list	Single-threaded use

CLASSIC USE CASE: Listener/Observer lists in UI frameworks and event systems. Listeners are registered rarely but events fire frequently. The list is iterated on every event dispatch and must be thread-safe. `CopyOnWriteArrayList` is a perfect fit: reads are free, writes are infrequent.

Section 5 — Iteration Patterns & ConcurrentModificationException

5.1 The modCount Mechanism

ArrayList (and most JCF classes) maintain an int field **modCount** that is incremented on every structural modification (add, remove, set size, clear, etc.). When an Iterator is created, it captures the current modCount as **expectedModCount**. On each call to next(), it checks: if modCount != expectedModCount, throw **ConcurrentModificationException**. This fail-fast behaviour is intentional — it catches programming bugs early rather than silently producing incorrect results.

```
// WRONG – ConcurrentModificationException on second iteration step
List<String> list = new ArrayList<>(Arrays.asList("a", "b", "c"));
for (String s : list) { // for-each uses Iterator internally
    if (s.equals("b"))
        list.remove(s); // modCount++ -> next call to it.next() throws CME!
}

// CORRECT METHOD 1: Iterator.remove()
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("b"))
        it.remove(); // uses expectedModCount, keeps them in sync
}

// CORRECT METHOD 2: removeIf (Java 8+) – BEST PRACTICE
list.removeIf(s -> s.equals("b"));

// CORRECT METHOD 3: collect to remove list, then bulk remove
List<String> toRemove = new ArrayList<>();
for (String s : list) {
    if (s.equals("b")) toRemove.add(s);
}
list.removeAll(toRemove);

// CORRECT METHOD 4: iterate backwards by index
for (int i = list.size() - 1; i >= 0; i--) {
    if (list.get(i).equals("b")) list.remove(i);
}
```

5.2 All Iteration Patterns

```
List<String> list = new ArrayList<>(List.of("A","B","C","D","E"));

// 1. Classic for-each (uses Iterator, read-only during iteration)
for (String s : list) System.out.println(s);

// 2. Index-based for loop – safe for reading; can modify via set(i, v)
for (int i = 0; i < list.size(); i++) System.out.println(list.get(i));

// 3. Iterator – safe removal with it.remove()
for (Iterator<String> it = list.iterator(); it.hasNext(); ) {
    String s = it.next();
    if (s.equals("C")) it.remove();
}

// 4. ListIterator – bidirectional + set(e) + add(e) during iteration
for (ListIterator<String> li = list.listIterator(); li.hasNext(); ) {
    String s = li.next();
    li.set(s.toLowerCase()); // replace current element
    if (s.equals("b")) li.add("inserted"); // insert after current
}

// 5. removeIf – Predicate-based bulk removal (preferred, readable, O(n) single pass)
list.removeIf(s -> s.length() > 1);

// 6. replaceAll – in-place transformation with UnaryOperator
list.replaceAll(String::toUpperCase); // replaces ALL elements in-place
```

```
// 7. Stream forEach – for side effects; do NOT modify source list inside  
list.stream().filter(s -> !s.isEmpty()).forEach(System.out::println);  
// 8. forEach (Iterable) – for-each with lambda, same as #1 but lambda syntax  
list.forEach(System.out::println);
```

replaceAll vs stream().map().collect(): replaceAll modifies the list in-place ($O(n)$, no new list allocation). stream().map().collect() creates a NEW list. Use replaceAll when you want in-place mutation; use stream+collect when you need the original preserved or want to return a different type.

Section 6 — Immutable & Unmodifiable Lists

6.1 Immutability Spectrum

```
// 1. Collections.unmodifiableList – wrapper/view, original still mutable
List<String> mutable = new ArrayList<>(List.of("a","b","c"));
List<String> unmod = Collections.unmodifiableList(mutable);
unmod.get(0); // OK – reads allowed
// unmod.add("x"); // throws UnsupportedOperationException
mutable.add("d"); // ALLOWED – unmod now shows 4 elements!
System.out.println(unmod); // [a, b, c, d] – NOT truly immutable

// 2. List.of() – truly immutable (Java 9+)
List<String> immutable = List.of("a","b","c");
// immutable.add("x"); // UnsupportedOperationException
// immutable.set(0,"y"); // UnsupportedOperationException
// immutable.add(null); // NullPointerException – nulls forbidden
// Iteration and reads are fine

// 3. List.copyOf() – defensive copy + immutable (Java 10+)
List<String> copy = List.copyOf(mutable); // copies current state, then immutable
mutable.add("e");
System.out.println(copy); // still [a, b, c, d] – independent snapshot

// 4. Collections.singletonList – single-element immutable view
List<String> single = Collections.singletonList("only");
// single.add("x"); // UnsupportedOperationException
// single.set(0,"y"); // UnsupportedOperationException

// 5. Arrays.asList – fixed-size, partially mutable
String[] arr = {"a","b","c"};
List<String> fixed = Arrays.asList(arr);
fixed.set(0,"CHANGED"); // OK – array element updated (and arr[0] changes too!)
// fixed.add("x"); // UnsupportedOperationException
// fixed.remove(0); // UnsupportedOperationException
arr[1] = "ALSO CHANGED"; // also changes fixed.get(1) – backed by same array!
```

6.2 Immutability Comparison

Method	Truly Immutable?	Null OK?	set() allowed?	add/remove?	Backed by original?
<code>new ArrayList<>()</code>	No	Yes	Yes	Yes	No — own copy
<code>Arrays.asList()</code>	No (partially)	Yes	Yes	No — fixed size	Yes — same array
<code>Collections.unmodifiableList()</code>	No (view)	Yes	No	No	Yes — wraps original
<code>Collections.singletonList()</code>	Yes (effectively)	Yes	No	No	N/A — internal
<code>List.of()</code>	Yes	No — NPE	No	No	No — internal copy
<code>List.copyOf()</code>	Yes	No — NPE	No	No	No — defensive copy

Section 7 — Performance Comparison

7.1 ArrayList vs LinkedList vs CopyOnWriteArrayList

Operation	ArrayList	LinkedList	CopyOnWriteArrayList
get(i)	O(1)	O(n)	O(1)
add(end)	Amort O(1)	O(1)	O(n) — full copy
add(mid)	O(n)	O(n) to find + O(1) splice	O(n) — full copy
remove(end)	O(1) amort	O(1)	O(n) — full copy
remove(mid)	O(n)	O(n) find + O(1) splice	O(n) — full copy
contains(o)	O(n)	O(n)	O(n)
iterator next()	O(1)	O(1)	O(1) on snapshot
ListIterator add/remove	O(n)	O(1)	Not supported (read-only)
Memory per element	~8B (ref)	~24B (Node: prev+next+val)	~8B (ref)
Thread safety	No	No	Yes (lock-free reads)
ConcurrentModificationException	Yes (fail-fast)	Yes (fail-fast)	Never
Cache locality	Excellent	Poor (scattered nodes)	Excellent

7.2 Decision Guide

Scenario	Best Choice	Reason
General-purpose mutable list	ArrayList	Best cache locality, O(1) random access, lowest overhead
Frequent adds/removes at both ends	ArrayDeque	O(1) at both ends, better cache than LinkedList
Need List + Deque as same object	LinkedList	Only class implementing both interfaces
Heavy mid-list ListIterator insertions	LinkedList	O(1) per ListIterator add/remove vs O(n) for ArrayList
Read-heavy concurrent list	CopyOnWriteArrayList	Lock-free reads, snapshot iterators, no CME
Write-heavy concurrent list	Collections.synchronizedList or ConcurrentLinkedDeque	COWAL O(n) writes too expensive
Known size, no structural changes	List.of() or Arrays.asList()	Zero overhead, immutable/fixed
Numeric data, performance critical	int[] or long[] (primitives)	No boxing overhead, better GC, cache-line dense

Q&A; — Standard Questions (Q1–Q35)

Q1. How does ArrayList grow internally? What is the growth factor and why 1.5× instead of 2×?

ArrayList starts at capacity 10 (lazily on first add). When full, $\text{newCapacity} = \text{oldCapacity} + (\text{oldCapacity} >> 1)$, which is 1.5× growth. The 1.5× factor balances amortised $O(1)$ appends (requires geometric growth) against memory waste. A 2× factor would waste up to 50% of allocated space; 1.5× wastes up to 33%. For large lists (millions of elements) this is significant. Python's list uses ~1.125× for even tighter memory control.

Q2. What is the difference between ArrayList and a regular Java array?

Arrays have fixed size set at creation; ArrayList grows dynamically. Arrays support primitives (`int[]`); ArrayList requires boxed types (`Integer`), causing autoboxing overhead. ArrayList provides rich API (`sort`, `removeIf`, `stream`). Arrays support covariant types (`String[]` is-a `Object[]`), which can cause `ArrayStoreException`. ArrayList uses generics for type safety. `Arrays.asList()` bridges the two worlds.

Q3. What are the 3 correct ways to remove elements during List iteration?

(1) `Iterator.remove()` — call `it.next()` first, then `it.remove()`; keeps `modCount` in sync. (2) `list.removeIf(predicate)` — Java 8+; single-pass, most readable, internally uses iterator. (3) Iterate backwards by index: `for (int i = list.size()-1; i >= 0; i--) if (...) list.remove(i);` — safe because removing at index `i` only shifts elements after `i`, not the ones you already processed.

Q4. What does subList() return? How do mutations affect the parent list?

`subList(fromIndex, toIndex)` returns a **view** backed by the original list. Mutations to the subList (`set`, `sort`, `clear`, `remove`) ARE reflected in the parent list. Adding to or removing from the parent list OUTSIDE the subList range after calling `subList()` causes `ConcurrentModificationException` on the next subList operation (because `modCount` changes). To get an independent copy: `new ArrayList<>(list.subList(1,3))`.

Q5. What Deque operations does LinkedList support? When should you use LinkedList as a Queue or Stack?

LinkedList implements Deque: `addFirst/Last`, `removeFirst/Last`, `peekFirst/Last`, `offerFirst/Last`, `pollFirst/Last`, `push`, `pop`. Prefer LinkedList as a combined List+Deque only when you need both interfaces on the same object reference. For a pure Queue, use `ArrayDeque` — no node allocation, better cache locality. For a pure Stack, also prefer `ArrayDeque`. The legacy Stack class extends `Vector` and is synchronized unnecessarily.

Q6. Why is LinkedList rarely the right choice over ArrayDeque for queue/deque operations?

`ArrayDeque` stores elements in a resizable circular array — all elements are contiguous in memory, so CPU cache lines are used efficiently. `LinkedList` scatters `Node` objects across the heap; traversal causes cache misses. `ArrayDeque` also avoids per-element object allocation overhead (no `Node` with `prev/next` refs). Benchmarks consistently show `ArrayDeque` 2-3× faster than `LinkedList` for queue operations. `LinkedList`'s only edge case advantage is $O(1)$ `ListIterator` `add/remove` in the middle.

Q7. What are the iterator semantics of CopyOnWriteArrayList? Can it throw ConcurrentModificationException?

`COWArrayList`'s iterator captures a snapshot of the backing array at the moment `iterator()` is called. It never throws `ConcurrentModificationException` because it never reads from the live array — only the snapshot. The snapshot is immutable, so `iterator.remove()` is NOT supported (throws `UnsupportedOperationException`). If another thread adds elements during iteration, the iterator will NOT see those new elements — it sees the state at iterator creation time.

Q8. What are the differences between List.of(), Arrays.asList(), and new ArrayList()?

`new ArrayList()`: fully mutable, allows null, own backing array, dynamic resize. `Arrays.asList(a,b,c)`: fixed-size (no `add/remove`), allows null, BACKED BY THE PASSED ARRAY (mutations shared), `set()` is allowed. `List.of(a,b,c)`: completely immutable (no `add/remove/set`), null elements throw `NPE`, not backed by any array (internal optimized implementation), Java 9+. Rule of thumb: `List.of()` for constants; `Arrays.asList()` when bridging arrays; `new ArrayList<>()` when you need mutation.

Q9. What is the difference between `Collections.unmodifiableList()` and `List.of()`?

`Collections.unmodifiableList()` wraps an existing list as a view — if the original list is modified, the unmodifiable view reflects those changes. It is NOT truly immutable. `List.of()` creates a completely independent, truly immutable list. No mutations are possible through any reference. Additionally, `List.of()` forbids null elements (NPE on null), while `unmodifiableList()` allows whatever the original allows. Use `List.of()` when you want a guarantee of immutability; use `unmodifiableList()` for API boundary protection.

Q10. Explain the `remove(int index)` vs `remove(Object o)` ambiguity for `List`.

`List` has two `remove` overloads: `remove(int index)` and `remove(Object o)`. For `List myList`, `myList.remove(1)` calls `remove(int)` — removes element AT index 1. To remove by VALUE for an Integer, you must box: `myList.remove(Integer.valueOf(1))`. This is a common bug. Example: `List list = new ArrayList<>(List.of(1,2,3)); list.remove(1)` removes index 1 (value=2), leaving [1,3]. `list.remove(Integer.valueOf(1))` removes the first occurrence of value 1, leaving [2,3].

Q11. What can `ListIterator` do that `Iterator` cannot?

`Iterator`: `hasNext()`, `next()`, `remove()`. Goes forward only. `ListIterator` extends `Iterator` with: `hasPrevious()`, `previous()`, `nextIndex()`, `previousIndex()`, `set(e)` (replace last returned element), `add(e)` (insert after current position). `ListIterator` is bidirectional — can traverse forward and backward. `add()` in `ListIterator` does NOT cause `ConcurrentModificationException` (unlike adding to the list directly) because it updates the `expectedModCount` after the structural modification.

Q12. What happens if you create `ArrayList` with initial capacity 0?

`new ArrayList<>()` creates the list with `EMPTY_ELEMENTDATA` (a shared empty array). On the first `add()`, it grows to capacity 1 (since $0 \gg 1 = 0$, but min growth ensures at least 1). Subsequent adds grow by 1.5× from 1: 1, 2, 3, 4, 6, 9, 13, 19, 28... This causes more frequent reallocations than the default capacity-10 start. Use `new ArrayList<>(0)` only when you expect the list to remain empty most of the time.

Q13. How do you sort a `List` with a custom `Comparator`? What algorithm does `Collections.sort` use?

`list.sort(comparator)` or `Collections.sort(list, comparator)`. Example: `list.sort(Comparator.comparingInt(Person::getAge).thenComparing(Person::getName));` `Collections.sort()` delegates to `list.sort()` which uses `Arrays.sort()` on the backing array. `Arrays.sort()` for objects uses **TimSort** — a hybrid of merge sort and insertion sort. TimSort is $O(n \log n)$ worst case, $O(n)$ best case (already sorted), and stable (preserves equal-element order). It exploits existing runs of sorted data (natural runs) for efficiency.

Q14. What is the performance difference between `contains()` in `ArrayList` vs `HashSet`?

`ArrayList.contains(o)`: $O(n)$ linear scan, calls `equals()` on each element until found or exhausted. `HashSet.contains(o)`: $O(1)$ average — computes `hashCode()`, maps to bucket, checks `equals()` on bucket's elements. For frequent membership testing: use a `HashSet` (or build a `HashSet` from your list). For ordered iteration with occasional `contains`: keep `ArrayList` for iteration, maintain a parallel `HashSet` for lookups. Note: `LinkedHashSet` gives insertion-ordered iteration WITH $O(1)$ `contains`.

Q15. What happens with concurrent modification of `ArrayList` from multiple threads?

`ArrayList` is NOT thread-safe. Concurrent structural modifications (`add/remove` from two threads) can cause: (1) Lost updates — one thread's `add` overwrites another's. (2) `ArrayIndexOutOfBoundsException` — size counter and array state become inconsistent. (3) `NullPointerException` — partial array state visible to other threads due to lack of memory visibility guarantee. (4) Infinite loops in rare cases. Solutions: synchronize externally, use `Collections.synchronizedList()`, `Vector`, or `CopyOnWriteArrayList`.

Q16. How does `Arrays.asList` handle primitive arrays? What is the gotcha?

`Arrays.asList()` does NOT work as expected with primitive arrays. `int[] arr = {1,2,3}; List list = Arrays.asList(arr);` — creates a `List` with ONE element (the `int[]` array). This is because `Arrays.asList()` takes a `T... vararg`; for `int[]` (a primitive array), `T` is inferred as `int[]` not `int`. To convert `int[]` to `List`, use Java 8+: `Arrays.stream(arr).boxed().collect(Collectors.toList());` or `IntStream.of(arr).boxed().toList()` in Java 16+.

Q17. What is the RandomAccess marker interface? Why does ArrayList implement it?

RandomAccess (java.util) is a marker interface (no methods) indicating that a list supports $O(1)$ positional access. ArrayList implements it; LinkedList does not. Algorithms in Collections utility class check for RandomAccess: if (list instanceof RandomAccess) { use index-based loop } else { use Iterator }. For example, Collections.binarySearch() and Collections.shuffle() use this check to choose the optimal strategy. A for-loop over LinkedList using get(i) is $O(n^2)$ but using Iterator is $O(n)$ — RandomAccess enables this distinction.

Q18. What happens if you structurally modify the parent list after calling subList()?

After list.subList(from, to) is called, any structural modification to the PARENT list (add, remove, clear — anything that changes size or modCount) causes the subList to become invalid. The next operation on the subList (get, add, size, iteration) throws ConcurrentModificationException. The subList itself detects this by checking the parent's modCount. This is a common gotcha: you cannot hold a subList reference across parent modifications.

Q19. What is the difference between replaceAll() and stream().map().collect()?

list.replaceAll(op): modifies the list IN PLACE using a UnaryOperator. $O(n)$ single pass, no new list allocated.
list.stream().map(f).collect(Collectors.toList()): creates a NEW list. Original is untouched. Use replaceAll when: you want in-place mutation, the list is already the target, memory allocation is a concern. Use stream+collect when: you need the original preserved, need a different collection type, or chaining multiple operations.

Q20. How does ArrayList.toArray() work? What overloads exist?

ArrayList has two toArray overloads: (1) Object[] toArray() — returns Object[]; cannot be cast to String[] even if list contains only Strings. (2) T[] toArray(T[] a) — pass a typed array; if large enough, fills it and returns it; if too small (common pattern: toArray(new String[0])), allocates a correctly-typed new array of the right size. Best practice: list.toArray(new String[0]) — new String[0] is cheap, signals the type, avoids oversized array. Java 11+ adds toArray(IntFunction generator): list.toArray(String[]::new).

Q21. What is the time complexity of ArrayList.clear()?

ArrayList.clear() is $O(n)$. It must null out every element reference in the backing array (elementData[i] = null for all $i < \text{size}$) to allow GC to reclaim the objects. Simply setting size=0 without nulling would cause memory leaks — the array would still hold references. The backing array capacity is NOT reduced; only the elements are cleared. To also reclaim the backing array, use trimToSize() after clear(), or just create a new ArrayList.

Q22. How does ensureCapacity() help performance?

ensureCapacity(minCapacity) pre-allocates the backing array if current capacity is less than minCapacity. Use it before bulk-loading: list.ensureCapacity(10000); for (Item i : src) list.add(i); This avoids multiple intermediate array copies during growth (log base 1.5 of 10000 $\approx$ 21 resizes avoided). Alternatively, pass the expected size to the constructor: new ArrayList<>(expectedSize). trimToSize() does the opposite — shrinks the backing array to exactly size, freeing wasted capacity.

Q23. Can ArrayList contain null elements? How does indexOf handle null?

Yes, ArrayList allows multiple null elements (unlike Set.of() or List.of()). indexOf(null) uses == null comparison: for (int i = 0; i < size; i++) if (o == null ? e == null : o.equals(e)). So indexOf(null) returns the index of the first null. remove(Object o) uses the same logic — correctly handles null without NPE. Note: sorting a list with null elements throws NullPointerException unless the Comparator handles nulls (use Comparator.nullsFirst() or Comparator.nullsLast()).

Q24. What is the difference between Iterator and Iterable?

Iterable (java.lang): interface with single method iterator() returning Iterator. Collections implement Iterable to enable for-each syntax. Iterator (java.util): interface with hasNext(), next(), and optional remove(). It is a one-time-use cursor — each call to iterator() on an Iterable creates a fresh cursor. Iterable = 'I can give you an iterator'; Iterator = 'I am currently pointing at a position in a sequence'. You can also make any custom class for-each-able by implementing Iterable.

Q25. What does `Collections.sort()` guarantee about stability?

`Collections.sort()` is guaranteed stable by the Java specification (since Java 1.2). Stability means elements that are considered equal (`compare()` returns 0) maintain their relative original order. This is critical for multi-key sorting: first sort by name, then sort by age — because sort is stable, the name ordering is preserved within each age group. TimSort (the underlying algorithm) is inherently stable. `Arrays.sort()` for `Object[]` is also stable; but `Arrays.sort()` for primitives (`int[]`, etc.) uses dual-pivot quicksort which is NOT stable.

Q26. How do you convert a List to a Set while preserving insertion order?

Use `LinkedHashSet`: `Set set = new LinkedHashSet<>(list)`; `LinkedHashSet` maintains insertion order (using a doubly-linked list alongside the hash table) while eliminating duplicates. $O(n)$ time and space. Alternatively: `Set set = list.stream().collect(Collectors.toCollection(LinkedHashSet::new))`; If order doesn't matter: `new HashSet<>(list)`. If sorted order: `new TreeSet<>(list)`.

Q27. What is the contract of `equals()` and `hashCode()` for List?

Two Lists are equal if they have the same size and all corresponding elements are equal (by `equals()`). `List.equals()` is defined in `AbstractList` and checks: (1) same object? return true. (2) other instanceof `List`? if not, false. (3) iterate both with iterators, check pairwise equals. `List.hashCode()` is defined as: `int h = 1; for (E e : this) h = 31*h + (e==null ? 0 : e.hashCode());` return h. Consequence: `ArrayList` and `LinkedList` with the same elements are `equals()` to each other and have the same `hashCode`.

Q28. How does `Collections.frequency()` work? What is it equivalent to?

`Collections.frequency(collection, obj)` counts how many elements equal `obj` using `equals()`. Returns `int`. Equivalent to: `(int) collection.stream().filter(e -> Objects.equals(e, obj)).count()`; Time complexity: $O(n)$ — linear scan. For counting multiple element frequencies, prefer: `Map freq = list.stream().collect(Collectors.groupingBy(e->e, Collectors.counting()))`;

Q29. What is fail-fast vs fail-safe iteration?

Fail-fast iterators (`ArrayList`, `HashMap`, `LinkedList`): throw `ConcurrentModificationException` if the collection is structurally modified after iterator creation. Detects via `modCount`. Fail-safe iterators (`CopyOnWriteArrayList`, `ConcurrentHashMap`): operate on a snapshot or use internal locks; never throw CME but may not see the latest data. The terms 'fail-fast' and 'fail-safe' are not in the Java spec — they are community terminology. The spec only guarantees that fail-fast behaviour is best-effort, not a guarantee.

Q30. How does `Collections.reverse()` work on a List? Is it in-place?

`Collections.reverse(list)` reverses the list IN PLACE. It uses a two-pointer swap approach: swap element at index `i` with element at index `(size-1-i)` for `i` from 0 to `size/2`. $O(n)$ time, $O(1)$ space. For `RandomAccess` lists, uses `list.set(i, list.set(j, list.get(i)))` which swaps via `set`'s return value. For sequential-access lists, uses a `ListIterator`-based approach. To get a reversed VIEW without modifying the original: use `List.copyOf` with `Collections.reverse` on a copy, or in Java 21+: `list.reversed()` returns a reversed view.

Q31. What is the difference between `addAll()` and `add()` in terms of performance?

`addAll(Collection c)` is significantly more efficient than repeated `add()` calls for `ArrayList`. `addAll()` calls `ensureCapacity` once for all elements, then uses `System.arraycopy()` for bulk copy. Repeated `add()` may trigger multiple `resize-and-copy` cycles. `addAll(index, c)` inserts a collection at a specific position: one shift to make room, then bulk copy. Always prefer `addAll()` when adding multiple elements from another collection.

Q32. How does `List.copyOf()` behave if the input list already is an immutable list?

`List.copyOf()` is optimized — if the input is already an unmodifiable list created by `List.of()` or `List.copyOf()`, it may return the SAME instance (no copy needed) since immutable objects are safe to share. This is an implementation detail and not guaranteed by the specification, but the JDK does this optimization. If the input is a mutable list (`ArrayList`), a defensive copy is always made.

Q33. What does `listIterator(index)` do?

`listIterator(int index)` returns a `ListIterator` positioned BEFORE the element at the given index. The next call to `next()` returns element at that index. The next call to `previous()` would require one step back (no element returned yet). Useful for starting iteration from a specific position — particularly efficient for `LinkedList` where it traverses from the nearest end (first if `index < size/2`, last otherwise).

Q34. What happens when you call `sort()` on an unmodifiable list?

Calling `list.sort()` or `Collections.sort()` on an unmodifiable list (from `Collections.unmodifiableList()`, `List.of()`, etc.) throws `UnsupportedOperationException`. Sorting requires `set()` operations on the list to perform swaps; unmodifiable lists reject `set()`. To sort immutable list content: collect to new `ArrayList`, sort that, optionally wrap again. List sorted = `new ArrayList<>(immutableList); sorted.sort(null);`

Q35. What is the difference between `size()` and `capacity()` in `ArrayList`?

`size()` is the number of actual elements stored; `capacity` is the length of the backing array. `ArrayList` has NO public `capacity()` method (unlike `StringBuilder.capacity()`). You can only access `capacity` via reflection in production code (not recommended). Initially: `size=0`, `capacity=0` (no-arg constructor, lazy init to 10 on first add). After 10 adds: `size=10`, `capacity=10`. 11th add: `size=11`, `capacity=15` ($10 + 10 >> 1 = 15$). `trimToSize()` sets `capacity = size` (releases unused slots).

Q&A; — Advanced Questions (Q36–Q52)

[ADV] Q36. Explain how ArrayList's internal array copy works at the JVM level. Why System.arraycopy instead of a loop?

ArrayList uses `Arrays.copyOf()` which delegates to `System.arraycopy()`. `System.arraycopy` is a native (JNI) method in the JVM, implemented in C/C++ using platform-optimized bulk memory copy instructions (like `memcpy` on Linux, or SIMD intrinsics on x86). A Java for-loop requires: fetch-each-index, bounds check, load element, store element, increment counter — all as JVM bytecodes with JIT overhead. `System.arraycopy` bypasses this loop entirely and copies a contiguous memory block in a single native call. For large arrays this is 10-100× faster. JIT can also intrinsify `System.arraycopy` — the JVM recognizes it and generates optimal native code directly.

[ADV] Q37. Describe the memory layout of ArrayList's Object[] and its impact on GC.

`Object[] elementData` contains object REFERENCES (4 bytes with compressed oops, 8 bytes without). The referenced objects (e.g., `String`, `Integer`) are scattered on the heap. A GC root scan traces through `elementData`, following each non-null reference to mark the referenced object. Because ArrayList holds strong references to all elements, elements cannot be garbage collected until removed. Large ArrayLists that hold old elements can cause 'logical memory leaks' — elements logically removed from business logic but still referenced (if you manually null-out by index but don't call proper `remove`). The JDK's `remove()` and `clear()` explicitly null `elementData` slots to break strong references.

[ADV] Q38. Why doesn't ArrayList.ListIterator.add() throw ConcurrentModificationException like adding to the list directly?

ArrayList's `Iter` (basic `Iterator`) checks `modCount == expectedModCount` on every `next()`. If they differ, it throws CME. `ArrayList.ListIter` (`ListIterator`) overrides `add(e)` to: (1) call `ArrayList.this.add(cursor, e)` — a structural mod, incrementing `modCount`. (2) Then immediately sets `expectedModCount = modCount` — resynchronizing. This is allowed because the `ListIterator` is the only entity modifying the list and it knows exactly what changed. The same logic applies to `ListIter.remove()` and `ListIter.set()`. Basic `Iter.remove()` also does this, but `add()` is exclusive to `ListIterator`.

[ADV] Q39. Why does Collections.sort use TimSort? Explain its time complexity guarantees.

TimSort was designed by Tim Peters for Python and adopted by Java (Java 7) for `Arrays.sort()` on `Object[]`. It is a hybrid of merge sort and binary insertion sort. Key properties: (1) Worst case $O(n \log n)$ — guaranteed, unlike quicksort's $O(n^2)$ worst case. (2) Best case $O(n)$ for already-sorted or nearly-sorted data — detects and merges existing sorted 'runs'. (3) Stable — equal elements maintain original order. (4) Adaptive — uses shorter insertion sort runs (minrun, typically 32-64) for small subarrays, then merges runs using a stack with carefully balanced merge criteria (galloping mode for long runs). Real-world data is often partially sorted (appended data, pre-sorted fields) making TimSort very practical.

[ADV] Q40. What is a 'structural modification' in JCF? Why does it matter for fail-fast semantics?

A structural modification is any operation that changes the SIZE of the list or otherwise modifies it in a way that iterations in progress may yield incorrect results. For ArrayList: `add`, `remove`, `clear`, `addAll`, `removeAll`, `retainAll`, `sort` (reorders elements, `modCount++`). NOT structural: `set()` — changes element value but not size or structure, does NOT increment `modCount` in ArrayList. `modCount` is incremented on every structural modification. Iterators capture it at creation (`expectedModCount`). On each `next()`, if `modCount != expectedModCount`, the iterator knows someone else modified the list and throws CME — catching multi-threaded and single-threaded programming bugs.

[ADV] Q41. How does CopyOnWriteArrayList's volatile array field ensure visibility without locking for reads?

The backing array reference is declared: `private transient volatile Object[] array`. The Java Memory Model guarantees that a volatile write HAPPENS-BEFORE any subsequent volatile read of the same field. When a writer thread calls `add()`: (1) acquires lock, (2) copies array, (3) writes new array reference to volatile field, (4) releases lock. The volatile write flushes all changes to main memory. When a reader thread calls `get()`: reads the volatile array reference — the JMM guarantees it sees the latest write. No lock is needed for reads because the read is a single atomic volatile load of the reference, and the referenced array is never mutated (a new copy is always created for writes).

[ADV] Q42. How is List.of() implemented internally in Java 11+? What is the ImmutableCollections.List12 vs ListN optimization?

Java 9 introduced `ImmutableCollections` (package-private). `List.of()` dispatches based on element count: `List.of()` -> `ImmutableCollections.emptyList()` (singleton); `List.of(e1)` -> `ImmutableCollections.List12` storing `e0` field only; `List.of(e1,e2)` -> `ImmutableCollections.List12` storing `e0` and `e1` fields; `List.of(e1,...,eN)` for $N > 2$ -> `ImmutableCollections.ListN` wrapping `Object[]` elements. `List12` avoids any array allocation for 1 or 2 elements — the elements are stored directly as fields. This saves an `Object[]` allocation + array header (~16 bytes) and one pointer indirection per access. For the very common case of small constant lists, this is a significant optimization.

[ADV] Q43. Why does ArrayList.remove(Object o) use == null check before calling equals()?

The `remove(Object)` implementation is: `if (o == null) { scan for null element } else { scan using o.equals(e) }`. This is necessary because `ArrayList` allows null elements. If you called `o.equals(e)` without the null check, you would get `NullPointerException` when `o` is null. Also note the direction: it calls `o.equals(e)`, NOT `e.equals(o)`. This matters because if `e` is null, calling `e.equals(o)` would also NPE. By always calling the non-null object's `equals()`, the code avoids NPE in both cases (null `o` and null `e`). The same pattern is used in `indexOf()` and `contains()`.

[ADV] Q44. How does ArrayList handle serialization? What is the significance of the custom writeObject?

`ArrayList` implements `Serializable` and has a custom `writeObject()/readObject()` pair. The `elementData` field is marked transient — it would NOT be serialized by default Java serialization. `writeObject`: writes the `ArrayList` metadata (size, `modCount` via `defaultWriteObject`), then iterates and writes only the ACTIVE elements (`elementData[0..size-1]`), not the full array capacity. This avoids serializing null slots beyond size. `readObject`: reads the size, then reads exactly size elements back into a fresh array. This means serialization is space-efficient: a 10-element `ArrayList` with capacity 100 serializes as 10 elements, not 100. A deserialized `ArrayList` has `capacity == size` (trimmed).

[ADV] Q45. What are the performance implications of boxing in List vs int[] for numeric workloads?

`List` stores `Integer` objects — each is a heap object (~16 bytes header + 4 bytes value = 20 bytes, aligned to 16 bytes = 16 bytes with compressed oops). The list itself stores references (4-8 bytes each). So 1M integers in `List`: ~1M references (4MB) + ~1M `Integer` objects (16MB) = ~20MB. `int[]`: 1M ints * 4 bytes = 4MB — 5× less memory, all in one contiguous block. Computation on `int[]` is cache-friendly (sequential access of adjacent 4-byte ints). `List` involves pointer chasing: load reference, follow to heap, load int — causing cache misses. For numeric-intensive algorithms (sorting, searching, summing millions of numbers), use primitive arrays or specialized collections like `IntStream`, `Trove IntArrayList`, or `Eclipse Collections IntLists`.

[ADV] Q46. Explain AbstractList.hashCode(). Why use the polynomial rolling hash with factor 31?

`AbstractList.hashCode()` computes: `int h = 1; for (E e : this) h = 31 * h + (e==null ? 0 : e.hashCode()); return h`. This is a polynomial rolling hash: $h = h * 31 + c$, evaluated across all elements. Why 31? (1) It is a prime — primes distribute bits more uniformly, reducing hash collisions. (2) Multiplication by 31 can be JIT-optimized as: $(h < 5) - h$ — a cheap shift and subtract. (3) It's large enough to spread values but small enough that 32-bit overflow wraps predictably. The same formula is used by `String.hashCode()` (over chars). The polynomial form ensures order matters: [1,2] and [2,1] have different hashCodes, which is correct since `equals()` also considers order.

[ADV] Q47. What Spliterator characteristics does ArrayList report and why does this matter for parallel streams?

`ArrayList.spliterator()` returns a `Spliterator` with characteristics: `ORDERED | SIZED | SUBSIZED`. `ORDERED`: elements have a defined encounter order (index order) — parallel operations must respect this. `SIZED`: size is known upfront — the framework can split work evenly without counting first. `SUBSIZED`: each split produces a `Spliterator` with a known size — enables recursive balanced splitting. These characteristics enable the parallel stream framework (`ForkJoinPool`) to split the list into roughly equal halves recursively: $[0, N/2)$ and $[N/2, N)$. Each fork processes its chunk, results are combined. With `SIZED+SUBSIZED`, the framework knows exactly how to split, avoiding over/under-partitioning. `LinkedList` only has `ORDERED` (no `SIZED` initially), making it less efficient for parallel streams.

[ADV] Q48. Why is LinkedList a poor choice for most use cases despite its $O(1)$ end-insertion? Explain cache locality.

LinkedList's theoretical $O(1)$ add/remove at ends is undermined by hardware reality. Each Node is a separate heap allocation. For a list of N elements, there are $N+1$ objects scattered across the heap. Modern CPUs load 64-byte cache lines. Sequential access of ArrayList reads ~8 references per cache line — each cache line load gives you 8 next-elements 'for free'. For LinkedList, each node.next pointer points to a Node that is likely NOT in the current cache line — each traversal step causes a potential L1/L2 cache miss (100-300 CPU cycles penalty each). For $N=1000$ LinkedList traversal: ~1000 cache misses. For ArrayList: ~125 cache misses. Additionally, per-node object headers and GC pressure from many small objects add overhead. Benchmarks on modern hardware show ArrayList often outperforms LinkedList even for add/remove at the middle.

Exercises — Part A: Coding Challenges

A1. Flatten a List<List<T>> without streams

```
// Implement: <T> List<T> flatten(List<List<T>> nested)
public static <T> List<T> flatten(List<List<T>> nested) {
    List<T> result = new ArrayList<>();
    for (List<T> inner : nested) {
        result.addAll(inner); // addAll uses System.arraycopy internally
    }
    return result;
}

// Alternatively, pre-calculate total size to avoid resizing:
public static <T> List<T> flattenOptimized(List<List<T>> nested) {
    int totalSize = 0;
    for (List<T> inner : nested) totalSize += inner.size();
    List<T> result = new ArrayList<>(totalSize); // pre-sized
    for (List<T> inner : nested) result.addAll(inner);
    return result;
}
```

A2. Thread-safe bounded list using CopyOnWriteArrayList

```
public class BoundedCOWList<E> {
    private final CopyOnWriteArrayList<E> list = new CopyOnWriteArrayList<>();
    private final int maxSize;

    public BoundedCOWList(int maxSize) { this.maxSize = maxSize; }

    // synchronized to make check-then-act atomic (CAS pattern)
    public synchronized boolean add(E element) {
        if (list.size() >= maxSize) return false;
        list.add(element);
        return true;
    }

    public E get(int index) { return list.get(index); } // lock-free read

    public int size() { return list.size(); }

    public Iterator<E> iterator() { return list.iterator(); } // snapshot iterator
}

// Note: add() must be synchronized – COWAL.add() alone is not atomic with the size check.
```

A3. Custom Iterator for a singly-linked list

```
class SinglyLinkedList<T> implements Iterable<T> {
    private Node<T> head;
    private int size;

    private static class Node<T> {
        T data;
        Node<T> next;
        Node(T data) { this.data = data; }
    }

    public void addFirst(T data) {
        Node<T> n = new Node<>(data);
        n.next = head; head = n; size++;
    }

    @Override
    public Iterator<T> iterator() {
        return new Iterator<T>() {
            private Node<T> current = head;

            @Override public boolean hasNext() { return current != null; }
        }
    }
}
```

```

@Override public T next() {
    if (!hasNext()) throw new NoSuchElementException();
    T val = current.data;
    current = current.next;
    return val;
}
// remove() not implemented (would require tracking previous node)
};
}
}

```

A4. Remove duplicates preserving insertion order without using Set

```

// Without Set: O(n^2) approach using indexOf
public static List<String> removeDuplicates(List<String> input) {
    List<String> result = new ArrayList<>();
    for (String s : input) {
        if (!result.contains(s)) // O(n) per call – total O(n^2)
            result.add(s);
    }
    return result;
}

// Efficient approach using LinkedHashMap (insertion-ordered, no duplicate keys)
public static List<String> removeDuplicatesEfficient(List<String> input) {
    // LinkedHashMap preserves insertion order while ensuring uniqueness
    Map<String, Boolean> seen = new LinkedHashMap<>();
    for (String s : input) seen.put(s, Boolean.TRUE);
    return new ArrayList<>(seen.keySet()); // O(n) total
}

// Java 8+ stream approach (uses LinkedHashSet internally):
// input.stream().distinct().collect(Collectors.toList())

```

A5. Rotate-left a List<T> by k positions in O(n)

```

// Rotate left by k: [1,2,3,4,5] k=2 -> [3,4,5,1,2]
// Classic 3-reverse approach: O(n) time, O(1) extra space
public static <T> void rotateLeft(List<T> list, int k) {
    int n = list.size();
    if (n == 0) return;
    k = k % n;
    if (k == 0) return;
    reverse(list, 0, k - 1); // reverse first k elements
    reverse(list, k, n - 1); // reverse remaining n-k elements
    reverse(list, 0, n - 1); // reverse whole list
}

private static <T> void reverse(List<T> list, int lo, int hi) {
    while (lo < hi) {
        T tmp = list.get(lo);
        list.set(lo, list.get(hi));
        list.set(hi, tmp);
        lo++; hi--;
    }
}

// Collections.rotate(list, -k) also works but that's 1 line – the question asks for manual impl

```

Exercises — Part B: Debug & Fix

B1. ConcurrentModificationException in for-each loop

```
// BUGGY CODE:
List<String> names = new ArrayList<>(Arrays.asList("Alice", "Bob", "Charlie"));
for (String name : names) {
    if (name.startsWith("B")) names.remove(name); // CME on next iteration!
}

// ROOT CAUSE: for-each uses an Iterator; calling names.remove() directly increments modCount
// but expectedModCount in the Iterator is stale. Next it.next() detects mismatch -> CME.

// FIX 1: Use Iterator.remove()
Iterator<String> it = names.iterator();
while (it.hasNext()) {
    if (it.next().startsWith("B")) it.remove();
}

// FIX 2: removeIf (Java 8+) – cleanest
names.removeIf(name -> name.startsWith("B"));

// FIX 3: Collect and removeAll
List<String> toRemove = names.stream().filter(n -> n.startsWith("B")).collect(Collectors.toList());
names.removeAll(toRemove);
```

B2. remove(1) vs remove(Integer.valueOf(1)) ambiguity

```
// BUGGY CODE – What does this print?
List<Integer> numbers = new ArrayList<>(Arrays.asList(10, 1, 20, 1, 30));
numbers.remove(1); // <-- Bug: removes by INDEX, not by VALUE!
System.out.println(numbers); // [10, 20, 1, 30] – removed element at index 1 (value=1)

// INTENT was probably: remove the first occurrence of value 1
numbers.remove(Integer.valueOf(1)); // remove by Object (value) – correct!
System.out.println(numbers); // [10, 20, 30]

// WHY: Java selects the most specific overload. remove(1) where 1 is an int literal
// matches remove(int index) exactly. Integer.valueOf(1) is an Integer, matching remove(Object o).

// SAFE PATTERN: always box when removing by value from List<Integer>
int valueToRemove = 1;
numbers.remove(Integer.valueOf(valueToRemove)); // unambiguous
```

B3. subList modification causing ConcurrentModificationException

```
// BUGGY CODE:
List<String> main = new ArrayList<>(Arrays.asList("a", "b", "c", "d", "e"));
List<String> sub = main.subList(1, 4); // view of [b,c,d]
main.add("f"); // structural modification to parent!
System.out.println(sub.get(0)); // CME thrown here!

// ROOT CAUSE: subList holds a reference to parent's modCount at creation time.
// main.add("f") increments parent modCount. sub.get(0) detects mismatch -> CME.

// FIX 1: Don't modify parent while holding a subList reference
// FIX 2: Make an independent copy before modifying parent
List<String> subCopy = new ArrayList<>(main.subList(1, 4)); // copy first
main.add("f"); // safe now – subCopy is independent
System.out.println(subCopy.get(0)); // "b" – works fine

// FIX 3: Clear via subList before modifying parent (if intent is to remove range)
main.subList(1, 4).clear(); // removes [b,c,d] from parent – valid
```

B4. Arrays.asList() UnsupportedOperationException

```
// BUGGY CODE:
String[] arr = {"apple", "banana", "cherry"};
```

```
List<String> list = Arrays.asList(arr);
list.add("date"); // UnsupportedOperationException!
list.remove("apple"); // UnsupportedOperationException!
// ROOT CAUSE: Arrays.asList() returns java.util.Arrays$ArrayList – a fixed-size list
// backed by the original array. The array cannot change size; add/remove not supported.
// set() IS allowed because it doesn't change size.
// FIX 1: Wrap in new ArrayList for full mutability
List<String> mutable = new ArrayList<>(Arrays.asList(arr));
mutable.add("date"); // OK!
// FIX 2: Use List.of() if immutability is acceptable (Java 9+)
List<String> immutable = List.of(arr); // also throws on add, but intended immutability
// GOTCHA: arr[0] = "CHANGED"; also changes list.get(0) – same backing array!
arr[0] = "MODIFIED";
System.out.println(list.get(0)); // "MODIFIED" – arrays.asList is a view over arr
// This DOES NOT apply to new ArrayList<>(Arrays.asList(arr)) – that makes a copy
```


Exercises — Part C: Multiple Choice Questions

MCQ 1. What is printed? `List<Integer> list = new ArrayList<>(Arrays.asList(1,2,3)); list.remove(2); System.out.println(list);`

- A) [1, 2]
- B) [1, 3]
- C) [2, 3]
- D) [1, 2, 3]

Answer: B — `remove(2)` is `remove(int index=2)`, removing the element at index 2 (value=3). Result: [1,2].

MCQ 2. Which List implementation should you use for a read-heavy, multi-threaded list that is rarely modified?

- A) ArrayList with synchronized block
- B) Vector
- C) CopyOnWriteArrayList
- D) `Collections.synchronizedList(new ArrayList<>())`

Answer: C — `CopyOnWriteArrayList` has lock-free reads (volatile array reference) and snapshot iterators. Best for read-heavy concurrent access.

MCQ 3. What does `new ArrayList<>(Arrays.asList(1,2,3)).subList(1,2)` return?

- A) [2, 3]
- B) [2]
- C) [1, 2]
- D) A copy of [2]

Answer: B — `subList(fromIndex, toIndex)` is `fromIndex` inclusive, `toIndex` exclusive. `subList(1,2)` returns a view containing only index 1 = [2].

MCQ 4. Which of the following will NOT throw `UnsupportedOperationException`?

- A) `List.of(1,2,3).add(4)`
- B) `List.of(1,2,3).set(0,9)`
- C) `Collections.unmodifiableList(new ArrayList<>(List.of(1))).get(0)`
- D) `Arrays.asList(1,2,3).add(4)`

Answer: C — `get()` is a read operation — allowed on any unmodifiable list. A, B, D all attempt mutations that throw `UnsupportedOperationException`.

MCQ 5. What is the output? `List<String> a = new ArrayList<>(List.of("x","y")); List<String> b = a; a.add("z"); System.out.println(b.size());`

- A) 2
- B) 3
- C) Throws CME
- D) Compile error

Answer: B — `a` and `b` reference the SAME `ArrayList` object. Adding to `a` is visible through `b`. `b.size()` = 3.

MCQ 6. For a `List<String>` with 1 million elements, which operation has the worst time complexity difference between `ArrayList` and `LinkedList`?

- A) `add(e)` at end
- B) `get(500000)`
- C) `size()`
- D) `iterator().next()`

Answer: B — `get(500000)` is $O(1)$ for `ArrayList` (direct array access) but $O(n)$ for `LinkedList` (must traverse ~500,000 nodes from the nearest end). The worst relative difference is at random index access.

Quick Reference Cheat Sheet

Topic	Key Points to Remember
Collection hierarchy	Iterable -> Collection -> List/Set/Queue; Map is separate (not-a Collection)
Why Map != Collection	Map.put(k,v) takes 2 args; Collection.add(e) takes 1; incompatible contracts
ArrayList growth	Default cap=10 (lazy); grows by 1.5x (oldCap + oldCap>>1); System.arraycopy (native)
ArrayList complexity	get/set O(1); add-end amort O(1); add/remove mid O(n); contains O(n)
subList()	Returns VIEW — mutations reflected in parent. Parent structural mod invalidates subList (CME)
LinkedList internals	Doubly-linked Node(prev,item,next); O(1) head/tail; O(n) random access; 24B/node overhead
LinkedList vs ArrayDeque	Prefer ArrayDeque for Deque — better cache locality, no node allocation overhead
CopyOnWriteArrayList	volatile array; lock-free reads; O(n) writes (full copy); snapshot iterator; no CME ever
ConcurrentModificationException	fail-fast: modCount vs expectedModCount. Safe fixes: Iterator.remove(), removeIf, backwards loop
Iterator vs ListIterator	ListIterator adds: hasPrevious/previous, set(e), add(e), nextIndex/previousIndex
removeIf vs remove loop	removeIf(pred): single-pass O(n), no CME risk — always prefer over loop+remove
replaceAll vs stream+collect	replaceAll: in-place O(n) no allocation. stream+collect: new list, original preserved
Immutability spectrum	ArrayList (full mutable) < Arrays.asList (fixed-size) < Collections.unmodifiableList (view) < List.of (truly immutable)
List.of() vs unmodifiable	List.of: truly immutable, no null, no mutation. unmodifiable: view, original still mutable through original ref
remove(int) vs remove(Object)	List: remove(1) removes index 1. remove(Integer.valueOf(1)) removes value 1. Always box for value removal!
RandomAccess marker	ArrayList implements it; LinkedList does not. Collections algorithms use instanceof check to pick O(n) vs O(n^2) strategy
TimSort	O(n log n) worst, O(n) best (pre-sorted), stable. Hybrid merge+insertion. Used by Collections.sort and Arrays.sort(Object[])
Boxing overhead	List: ~20B per element, heap-scattered. int[]: 4B per element, contiguous. Use primitives for numeric-intensive code

End of Sub-Chapter 1.2 — Collections Framework & List Implementations

Next: Sub-Chapter 1.3 — Set Implementations